Mediation in Cells, Not in Orchestrators: Composition of Orchestration Layer Distinctions and AI-as-Substrate-Mediator in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 6, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a single architectural property, the emergent consequence of composing two CKS commitments — orchestration layer distinctions (the architectural distinctions between the substrate-cell layer and the workflow-engine, agent-framework, and control-plane layers, specified at source paper §2.1, §4.3, §4.4, and §8.1) and AI-as-substrate-mediator (the LLM role specified at §4.1 and §4.2). The composition produces an architectural property neither commitment yields alone, and that property is what this note names.

Abstract

The CKS pattern's orchestration-layer commitment establishes that workflow engines, agent frameworks, and control planes are architecturally distinct from cells; the AI-as-substrate-mediator commitment establishes the role LLMs play within cells. Each commitment is well-specified on its own. Their composition produces a third architectural property — *orchestration-mediator separation* — that neither commitment yields independently: AI mediation lives within cells, orchestration logic is distinct from cell internals, workflow engines trigger cells without hosting the LLM mediation that occurs inside them, and CKS is not an agent framework. This note formalizes the composition as a standalone architectural property, articulates its four operational components, identifies what the composition forces beyond either commitment alone, names the anti-patterns it specifically rules out, and provides an operational test with three sharpening properties. The composition is consequential because the dominant 2024–2026 alternative — placing an LLM at the orchestration layer to make decisions about what to orchestrate — satisfies neither commitment in the CKS sense, and many deployments that satisfy one of the two commitments fail the composition.

1. Why this composition pair needs to be formalized as standalone

CKS commits at the coordination-knowledge layer; the source paper §2.1 establishes the substrate-cell distinction as the architectural locus, and §4.3 places three other layers (reasoning execution, coordination mechanisms, control plane) around it. The orchestration-layer derivation note (Li, 29 April 2026) extends that framing to specify that workflow engines and agent frameworks belong to the coordination-mechanisms layer and that none of the three adjacent layers substitute for the coordination-knowledge layer. The AI-as-substrate-mediator derivation note (Li, 24 April 2026) specifies the LLM's architectural role as a five-property mediator: it reads from substrate content as primary state, writes under human-authored orchestration rules, holds no substrate-relevant state outside the substrate, exercises no authority over substrate content, and produces writes recorded with attribution.

Each commitment is sufficient on its own to address the question it answers. Orchestration layer distinctions answer *which architectural layers exist and where CKS sits*. AI-as-substrate-mediator answers *what role LLMs play within the CKS layer*. Neither, alone, answers a third question that the composition forces: *where, across the layers, is the LLM mediation specifically located, and what relationship does that location have to the orchestration logic that surrounds it?* The two layers established by the orchestration-layer commitment are architecturally distinct; the mediator role specified by AI-as-substrate-mediator could in principle attach to either. The composition is what fixes the attachment: the mediator role lives at the coordination-knowledge layer, specifically inside cells, and not at the coordination-mechanism layer.

The motivating context is that the dominant 2024–2026 deployment shape in AI-mediated coordination places the LLM at the orchestration layer — under marketplace labels including "agent orchestrator," "AI-driven workflow," and "autonomous agent system" — sharing one architectural property: the LLM decides, at run time, what work to sequence. The source paper grounds the composition at §4.4, where the AI-as-substrate-mediator role is positioned within the four-layer landscape and differentiated from coordination-mechanism-layer commitments. This note operationalizes that positioning as a standalone derivation, opening the A1.15 composition cluster (the fourth tier of Phase A4 in the CKS derivation-note series).

2. The emergent architectural property: four operational components

In the CKS pattern, the composition produces an emergent architectural property called **orchestration-mediator separation**, defined by four operational components that hold jointly at all times during the substrate's existence.

Component 1 — AI mediation lives within cells. The five mediator properties (substrate-content reads as primary state; writes under human-authored orchestration rules; no substrate-relevant state outside the substrate; no authority over substrate content; LLM writes recorded with attribution) are exercised at the cell scope. The cell is the unit at which orchestration rules are authored and at which the LLM operates under them. AI mediation outside cells — in the workflow engine, in the agent framework's coordination layer, in the control plane — is not the mediator role and is governed by

different rules (or none).

Component 2 — Orchestration logic is distinct from cells. Orchestration logic (step sequencing, branching, retries, parallelism, multi-agent communication, scheduling, gateway-layer access decisions) operates at layers architecturally distinct from cells. The composition forces this distinction to hold *in the presence of LLMs* — that is, the introduction of LLMs into a deployment does not collapse the layer distinction, and the LLM does not become the orchestration logic. Orchestration logic is rule-driven: workflow engines execute their rules, agent frameworks execute their coordination semantics, control planes execute their gateway policies. None of these is the cell, and the LLM mediation is not at any of them.

Component 3 — Workflow engines trigger cells without hosting AI mediation. The composition pattern from the orchestration-layer note's §6 — "a workflow engine triggers a CKS cell" — is constrained so that the engine's role is triggering only. The engine decides *when* the cell should run given workflow state and triggering conditions; the LLM mediation that occurs once the cell starts is internal to the cell, governed by orchestration rules that are part of the cell's specification rather than the engine's. The engine records that it ran the cell; the substrate records what the cell decided. The mediator role does not move into the engine when an engine is present.

Component 4 — CKS is not an agent framework. Cells are not agents in the agent-framework sense. An agent, in the pattern the AI-as-substrate-mediator derivation distinguishes the mediator from, holds goals, plans, and intermediate state across multiple steps and exercises judgment about what actions to take next. A cell is a rule-governed processing unit in which an LLM operates as mediator under orchestration rules. The LLM's reasoning within a single cell invocation is admissible; the cell does not become an agent because its LLM reasoned, because the reasoning operates under rules and produces substrate writes recorded with attribution rather than autonomous action over a workflow.

A deployment that satisfies all four components implements the composition; a deployment that fails any one of them fails the composition, even if it satisfies one or both of the foundational commitments individually.

3. What the composition forces beyond either commitment alone

Orchestration layer distinctions alone admit deployments in which the layer distinctions are correctly drawn but AI is placed at one of the orchestration layers. A workflow-engine-plus-substrate deployment can preserve the strict layer distinction — the engine handles execution state, the substrate handles coordination state — and still place an LLM at the workflow-engine layer to determine which branches the engine takes. The layer distinction is preserved; the LLM is at the wrong layer. The orchestration-layer commitment alone does not rule this out, because it names the layer distinctions but does not specify where, within them, LLM mediation lives.

AI-as-substrate-mediator alone admits deployments in which the mediator role is correctly implemented within a unit called a "cell" but the orchestration logic surrounding the cell is itself AI-driven. The LLM inside the cell may satisfy the five mediator properties, and a separate LLM at the orchestration layer may decide which cells to run, in what order, with what inputs, by reasoning over

workflow state. The mediator role is correctly bounded inside the cell; the orchestration around the cell is AI-driven rather than rule-driven. The mediator commitment alone does not rule this out, because it specifies the LLM's role within a scope but not the architectural relationship between that scope and the orchestration that surrounds it.

The composition forces three things neither commitment alone forces. It forces *AI placement to be settled*: the mediator role attaches at the cell scope, and only at the cell scope. It forces *orchestration to be rule-driven, not AI-driven*: the orchestration logic that triggers cells, sequences them, branches between them, and handles retries is determined by engine rules, framework coordination semantics, or control-plane policies, because there is no mediator role available to occupy the orchestration layer. It forces *workflow engines to be triggers, not hosts*: they sequence cell invocations; the cells host the mediation.

4. What the composition is not

The composition is precise about what orchestration-mediator separation is. It is equally important to say what it is not, because each of the following is a real architectural pattern in adjacent literature, and conflating any of them with the composition produces a misreading.

Not orchestration layer distinctions alone. A deployment that maintains the layer distinction between coordination knowledge and coordination mechanisms but places an LLM at the coordination-mechanism layer satisfies the layer-distinction commitment in the strict sense and fails the composition. The "AI orchestrator" pattern is the canonical instance: an LLM at the workflow-engine or agent-framework layer makes decisions about what cells (or other units) to invoke. The layer distinction is preserved; the AI is at the wrong layer.

Not AI-as-substrate-mediator alone. A deployment that implements the mediator role within an entity called a cell but builds its surrounding orchestration around AI-decided routing satisfies the mediator commitment within the cell and fails the composition. The cell-internal mediator is correctly bounded; the AI-driven orchestration around the cell is not what the composition admits.

Not "AI-managed orchestration." The pattern in which an LLM manages workflow-engine state, decides retries, determines branching at run time, and sequences agent communication is at the coordination-mechanism layer and is not the mediator role. The composition rules it out as CKS-coherent regardless of whether the system also includes a substrate at the coordination-knowledge layer.

Not "agent-orchestrator framework." The pattern in which a top-level LLM agent orchestrates other LLM agents — selecting sub-agents, routing tasks, accumulating multi-agent results — is the canonical agent-framework pattern the orchestration-layer commitment distinguishes CKS from, and the canonical autonomous-agent pattern the mediator commitment distinguishes the mediator role from. The composition rules it out on both grounds.

Each of these is a useful pattern in some other architecture. None of them is the composition.

5. Anti-patterns specifically violating the composition

Five anti-patterns in the CKS series specifically violate the composition. Each is named here in terms of which composition component it fails.

LLM-as-autonomous-agent over substrate is the canonical composition violation. An LLM placed at the orchestration layer holds goals, plans, and intermediate state across multiple cells and exercises judgment about what cells to run. Component 1 fails (mediation is not within cells); Component 2 fails (orchestration is AI-driven); Component 4 fails (the system is, in effect, an agent framework with cells subordinated to it). This is the failure mode the composition most directly forecloses.

LLM-as-terminal-producer fails Component 1: an LLM produces an output that the orchestration layer treats as terminal — passed downstream without cell-internal mediation. Mediation does not occur inside a cell; the LLM operates as a producer at the orchestration layer rather than as a mediator under cell-scope rules.

The AI-orchestrator pattern — a non-CKS-specific name for the broader category in which AI lives at the orchestration layer making decisions about workflow execution — fails Component 2 directly. Whether or not cells exist, and whether or not a substrate is present, the placement of AI at the orchestration layer is the violation.

Cell-as-agent fails Component 4: cells are treated as agents that hold goals, plans, and intermediate state across invocations and exercise judgment about what to do next. The cell is not a rule-governed processing unit but an agent. The LLM inside the cell is not bounded to mediator-under-rules behavior. This is the canonical violation of the not-an-agent-framework commitment, instantiated through a redefinition of what "cell" means.

Workflow-as-AI-decision-tree fails Components 2 and 3: a workflow engine's branching is determined at run time by LLM-produced control flow rather than by the engine's rules. The engine is hosting AI mediation, not just triggering cells; orchestration is AI-driven. The deployment may include cells that satisfy the mediator role internally; the engine's role has nonetheless slipped from triggering to hosting.

A deployment that exhibits any of these patterns may satisfy individual CKS commitments and may serve other architectural purposes well; it does not implement the composition.

6. Operational test

A system implements orchestration-mediator separation if and only if all of the following hold at all times during the substrate's existence:

1. Every LLM operation that produces or modifies substrate-relevant state occurs within a cell, under orchestration rules authored by humans, and produces writes recorded in the substrate with attribution.
2. Orchestration logic across the deployment — step sequencing, branching, retries, parallelism, multi-agent communication where present, scheduling, gateway-layer policy decisions — is determined by workflow-engine rules, agent-framework coordination semantics, or control-plane

policies, not by LLM-produced control flow at the orchestration layer.

3. Workflow engines that exist in the deployment trigger cells without hosting LLM mediation. Their state is execution state; the substrate records what cells decided.
4. The deployment is not constructed as an agent framework with cells as sub-agents. Cells are rule-governed processing units, not agents holding goals, plans, or intermediate state across invocations.

Three sharpening properties refine the test for cases where the surface architecture is ambiguous.

(e.1) AI-cell-internal-placement. For every LLM operation, identify the cell within which the operation occurs and the orchestration rule under which the LLM is operating. If an LLM operation cannot be located inside a cell — or if it occurs at the workflow-engine, agent-framework, or control-plane layer — the deployment fails (e.1).

(e.2) Orchestration-rule-driven. For every orchestration decision (which step runs next, which branch is taken, whether to retry, which sub-flow is invoked), identify the rule, schedule, or policy that determined it. If any orchestration decision is determined by LLM-produced reasoning at the orchestration layer rather than by a rule the engine, framework, or control plane is executing, the deployment fails (e.2).

(e.3) Workflow-engine-trigger-only. For every workflow engine, verify that its role with respect to cells is triggering — deciding when the cell should run and passing inputs the cell's specification calls for — and not hosting. The engine may record execution state and may handle retries, branching, and parallelism per its rules; it may not contain the LLM mediation that the cell hosts. If a workflow engine contains LLM operations that produce or modify substrate-relevant state, the deployment fails (e.3).

A one-sentence summary test: a deployment implements the composition if AI mediation can be located precisely at one or more cells, the orchestration layer can be described without reference to LLM decision-making, and the workflow engines (if any) can be described as triggering cells without containing the cells' mediation.

7. Why naming this composition as standalone matters

Orchestration-mediator separation is the architectural property that distinguishes CKS from the dominant 2024–2026 alternative shape in AI-mediated coordination. Many deployments labeled "agent system," "AI orchestrator," "autonomous workflow," or "agent framework" satisfy neither commitment in the CKS sense, but the failure is easier to name as a single composition violation than as two separate commitment violations: AI is at the wrong layer. Formalizing the composition as a standalone architectural property gives downstream work a single citation target for that failure mode rather than requiring it to read both foundational commitments together each time.

The composition is also load-bearing for downstream commitments. Governance-of-AI-through-rules — the cross-cutting consequence of human-governed composed with AI-as-substrate-mediator, formalized at the A4 cluster's earlier governance pair — depends on the composition for its operational scope: rules govern AI behavior precisely because AI is cell-internal, and rules cannot govern AI

behavior at the orchestration layer because the composition forecloses placing AI there. The composition is therefore the architectural prerequisite that makes governance-of-AI-through-rules implementable rather than aspirational.

This note opens the A1.15 composition cluster, the fourth tier of Phase A4 in the CKS derivation-note series. Subsequent notes in the cluster will formalize three further compositions involving orchestration layer distinctions: an orchestration-retraceability note (workflow-engine activations and cell executions jointly retraceable under the path-retraceability commitment with the layer distinction preserved); an orchestration-source-of-truth note (workflow logic is not authoritative content; substrate-recorded coordination state is); and a closing composition-coherence note (consistency between layer distinctions and the three hybrid-composition patterns under the source paper's composition-requirements treatment). The fifth and final tier of Phase A4 then operationalizes the three hybrid-composition patterns and closes the phase.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Mediation in Cells, Not in Orchestrators: Composition of Orchestration Layer Distinctions and AI-as-Substrate-Mediator in the Coordination Knowledge Substrate Pattern.* May 6, 2026. ORCID: 0009-0004-8065-3235.